

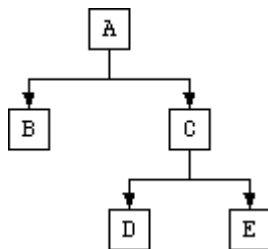
Kapitel 16. Multipel ärvning

Detta kapitel behandlar ett koncept som heter *multipel ärvning*, vilket innebär att klasser ärver från multipla basklasser.

Vad är multipel ärvning

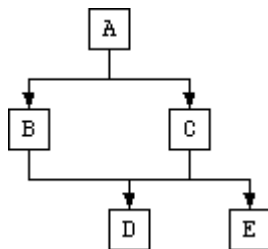
Med *multipel ärvning* avses möjligheten att ärva från flera olika basklasser. Normalt ärver man från endast en basklass och bygger upp en arvshierarki enligt nedanstående figur:

Figur 16-1. Normal arvshierarki



Här ärver alla klasser normalt singularärt. I den modifierade bilden nedan ärver D *både* B och C. D är alltså både en instans av klassen B och C:

Figur 16-2. Multipel arvshierarki



Multipel ärvning används då en ny klass skall representera två eller flera olika andra klasser samtidigt. Vi kan anta att vi har vår gamla klasshierarki från [Kapitel 15](#), där vi introducerade ett företag *Foo Factory* med en arvshierarki av anställda enligt:

```
class Employee { ... };  
class Manager : public Employee { ... };  
class Secretary : public Employee { ... };
```

Om vi tänker oss att företaget även har en *Technician*-class enligt:

```
class Technician : public Employee { ... };
```

Plötsligt har det uppkommit ett behov för en ny typ av anställd, en person som är chef, men har en teknisk kunskap. Man vill ha en person som är både *Manager* och *Technician*. Lösningen kan i så fall vara att använda multipel ärvning för att skapa denna nya klass:

```
class ManagerTechnician : public Manager, public Technician { ... };
```

För att deklarera att en klass ärver en annan multipelt placerar man alltså flera arvsdeklarationer efter varandra separerade av ett kommatecken (,). Man kan ärva från flera klasser än två ifall det finns behov för det. Man kan även ärva klasserna med olika skyddsnivå, även om vårt exempel ärver båda `public`. Nu har vi en ny klass `ManagerTechnician` som innehåller alla metoder från både `Manager` och `Technician`.

Problem med multipel ärvning

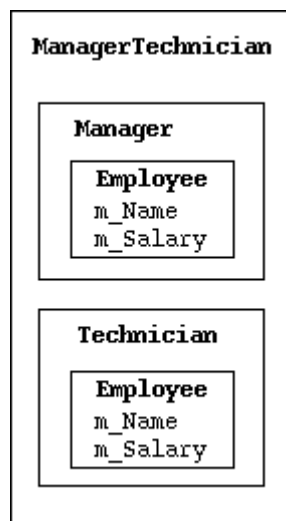
Vad finns det då för problem med multipel ärvning? Någotting måste det finnas som gör att metoden används så sällan? Det finns två stora problem som man med stor sannolikhet stöter på, nämligen:

- hur många objekt av basklasserna finns det egentligen? I vårt exempel ärver vi både `Manager` och `Technician`, alltså finns det två olika instanser av `Employee` som ligger i grund för det hela.
- vilken metod körs om samma metod finns i flera olika basklasser? Om våra exempelklasser har metoden `work()` som utför något specifikt för den typen av arbetare, vilken metod skall köras för `ManagerTechnician`? I vårt exempel vill vi sannolikt överlagra metoden `work()`, men om man inte gör det, vad händer då?

Antalet objekt

Vi kan illustrera problemet med flera objekt i den multipelt ärvda klassen med följande figur:

Figur 16-3. Flera objekt i multipel ärvd klass



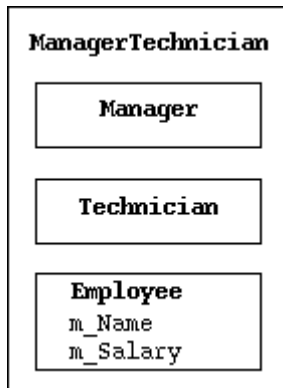
Eftersom både `Manager` och `Technician` ärver `Employee` finns denna klass "dubbelt" i `ManagerTechnician`. Det är nog inte meningen, för en dylik arbetstagare har då t.ex. två namn och två olika löner. Vi skulle vilja att `Employee` skulle kunna vara delad på något sätt. Lösningen på detta problem är att använda sig av *virtuella basklasser*. Vi bör då göra `Employee` till en virtuell basklass för både `Manager` och `Technician`. Nyttan med det är att vid multipel ärvning kommer endast en instans av en virtuell basklass att inkluderas. Vi måste då definiera om våra klasser en aning:

```
class Manager : virtual public Employee { ... };
```

```
class Technician : virtual public Employee { ... };
```

Vi har alltså lagt till nyckelordet `virtual` till klassdefinitionen. Om man ärver multiplet från klasser som alla har en virtuell basklass får man endast en instans av basklasserna, vilket är vad man normalt vill åstadkomma. Vi har nu följande situation:

Figur 16-4. Virtuella basklasser



Klassen `Employee` finns med endast en gång, om personen behöver inte längre lida av personlighetsklyvning. Om man blandar olika typer av arv får man dock ännu samma problem. Om t.ex. `Manager` inte är virtuellt ärvd från `Employee` kommer `ManagerTechnician` att innehålla två instanser av `Employee` igen. Två instanser av samma objekt ger oss problem om vi t.ex. har en metod som skriver ut löner på arbetare:

```
void paySalary (const Employee & Person) {
    ...
}
```

Vi använder alltså möjligheten att ett objekt alltid kan refereras till via sin basklass. Vilket blir i det senaste fallet det objekt som skickas till `paySalary()`, blir det `Employee`-objektet från `Manager`, eller `Employee`-objektet från `Technician`? Använd därför virtuella basklasser med förstånd där de behövs.

Vilken metod?

Om två klasser som ärvs multipelt av en tredje, och båda har samma metod definierad, vilken metod används då metoden exekveras? Här torde kompilatorn ge ett felmeddelande, eftersom den inte kan vet vilken metod som avses. Man måste då explicit ange vilken metod som avses. Om `Employee` har metoden `work()` så har `ManagerTechnician` samma metod tre gånger definierad, d.v.s. från `Manager`, `Technician` och ursprungsdefinitionen från `Employee` (om det inte var en rent virtuell metod). För att välja den metod som körs bör man använda syntaxen:

```
klassnamn::metod(parametrar);
```

för att explicit ange klassen vars metod skall användas. I vårt fall t.ex.:

```
void ManagerTechnician::doSomething () {
    // utför aktivt ledarskap
    Manager::work ();
    ...
    // utför teknikerarbete
}
```

```
    Technician::work ();  
}
```

Diskussion

Vi har sett att det finns en del problem med att använda multipel ärvning i praktiken. Man använder i normala fall mycket sällan multipel ärvning, eftersom de associerade problemen är ganska allvarliga. Purister inom objektorientering anser att multipel ärvning är "evil" och borde helt plockas bort ur C++. I t.e.x. Java finns ingen multipel ärvning såsom i C++, även om man där kan på ett vettigt sätt emulera fördelarna med multipel ärvning via `interface`. För det mesta lönar det sig att tänka om sin klasshierarki om det visar sig att man borde använda multipel ärvning. Exempel i fortsättningen kommer inte att använda sig av multipel ärvning.

[Förutgående](#)

Referera till basklasser

[Hem](#)

[Nästa](#)

Mera om klasser